

Revisit to CPE233 (Digital Design)

Authors:
WhisperingRock

Professor:

Class
Quarter
July 13, 2026

Contents

- 0.1 Hardware 2
- 0.2 Software 3
 - 0.2.1 A1 - Intro to Assembly Lang 3
- 0.3 Appendix 12

Note : All assignments are in the **CPE 233 Lab Manual**.

0.1 Hardware

0.2 Software

0.2.1 A1 - Intro to Assembly Lang

Note : Assume caller/callee does not need to preserve their respective registers in order to use them. KISS method applied for assignment 1.

Part 1

"Read three 16-bit unsigned values consecutively from SWITCHES (address 0x11000000), add them together, and output the result to LEDS (address 0x11000020). Assume the input values are 16-bit unsigned values. (Assume it is possible for the switches to change between each time they are read)"

Assuming the LEDS input value is limited to 16-bit unsigned as well.

Flowchart

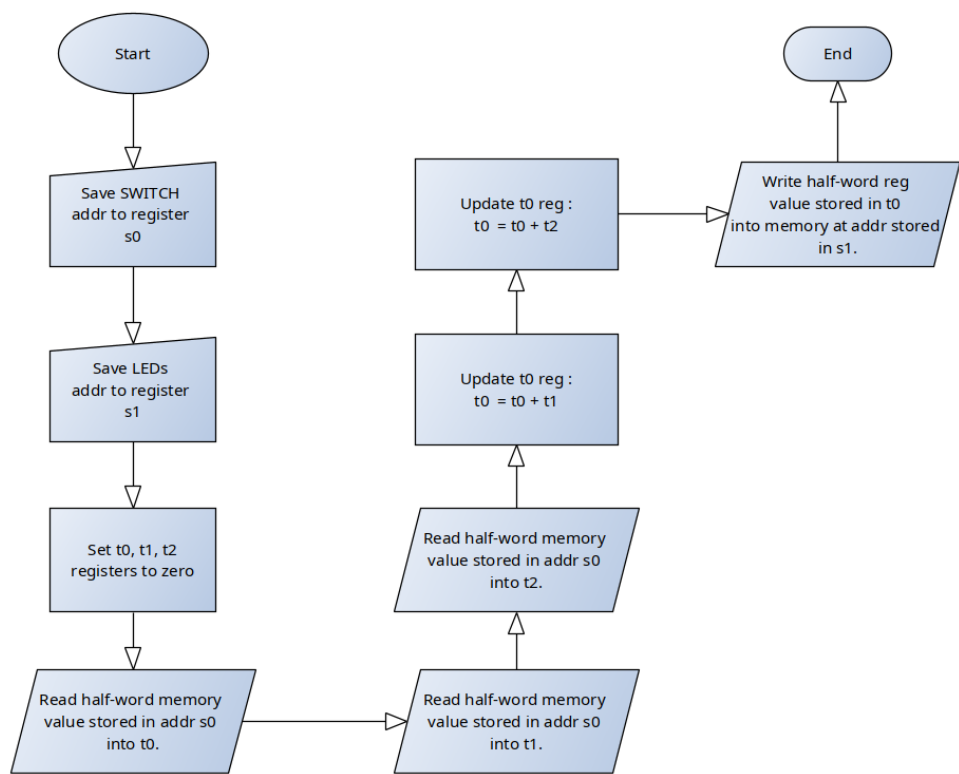


Figure 1: Flow Chart for Part 1

Verification Simulations

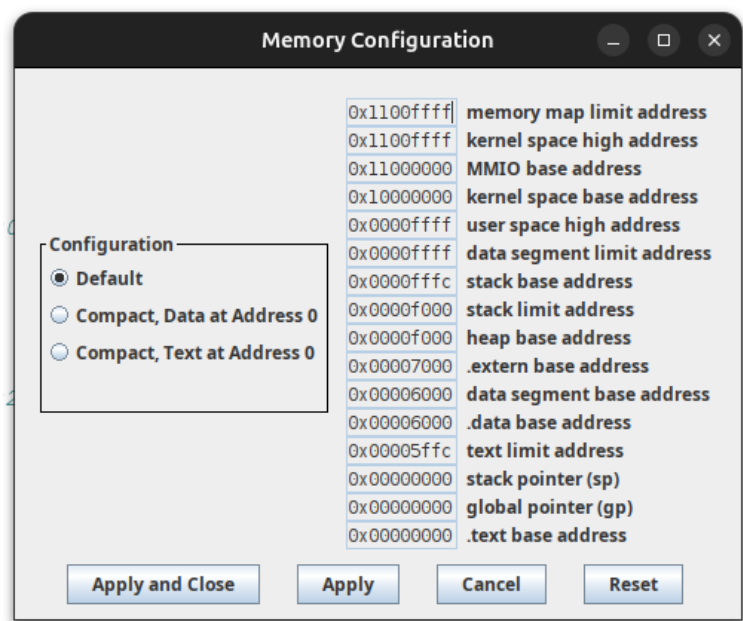


Figure 2: RARS Memory Configuration

```

20
21 #.equ SWITCHES_H,      0x11000 # upper 20 bits
22 #.equ SWITCHES_L,      0x000    # lower 12 bits
23 .equ SWITCHES_H,      0x00006 # upper 20 bits (data seg at 0x00006000)
24 .equ SWITCHES_L,      0x000    # lower 12 bits
25
26 #.equ LEDS_H,          0x11000 # upper 20 bits
27 #.equ LEDS_L,          0x020    # lower 12 bits
28 .equ LEDS_H,          0x00006 # upper 20 bits (data seg at 0x00006020)
29 .equ LEDS_L,          0x020    # lower 12 bits
30

```

Figure 3: Address Correction to work with RARS

- Test Case 1

```

33 # TC1 : Word Clip
34 .equ READ1_H,          0x00001 # switch first read (0x1234)
35 .equ READ1_L,          0x234
36
37
38 .equ READ2_H,          0x12345 # switch second read (0x5678)
39 .equ READ2_L,          0x678
40
41
42 .equ READ3_H,          0x11111 # switch third read (0x1111)
43 .equ READ3_L,          0x111
44 #   answer= 0x79BD

```

Figure 4: TC1 given inputs

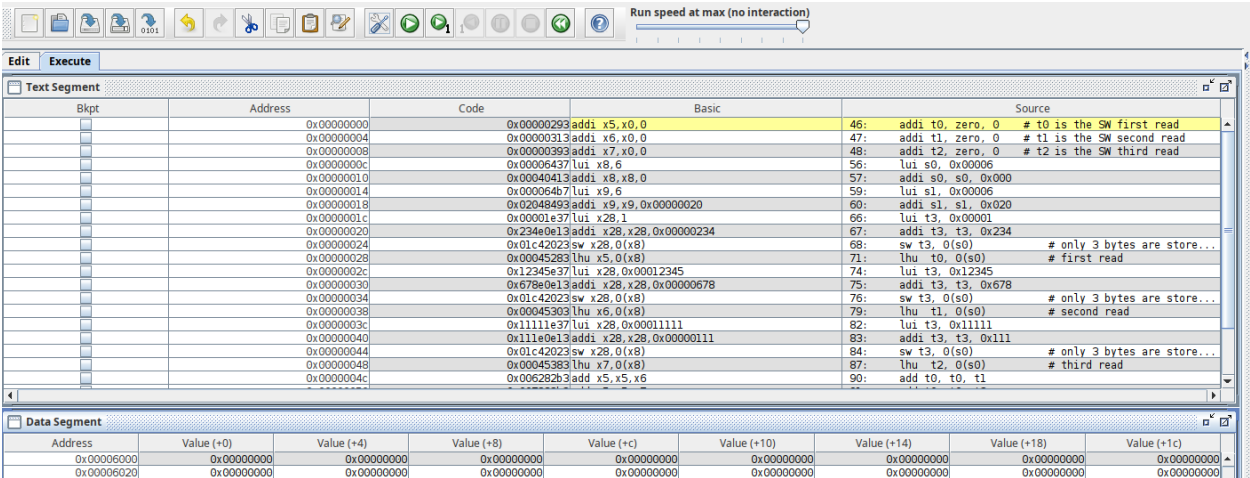


Figure 5: Data segment before executing instructions

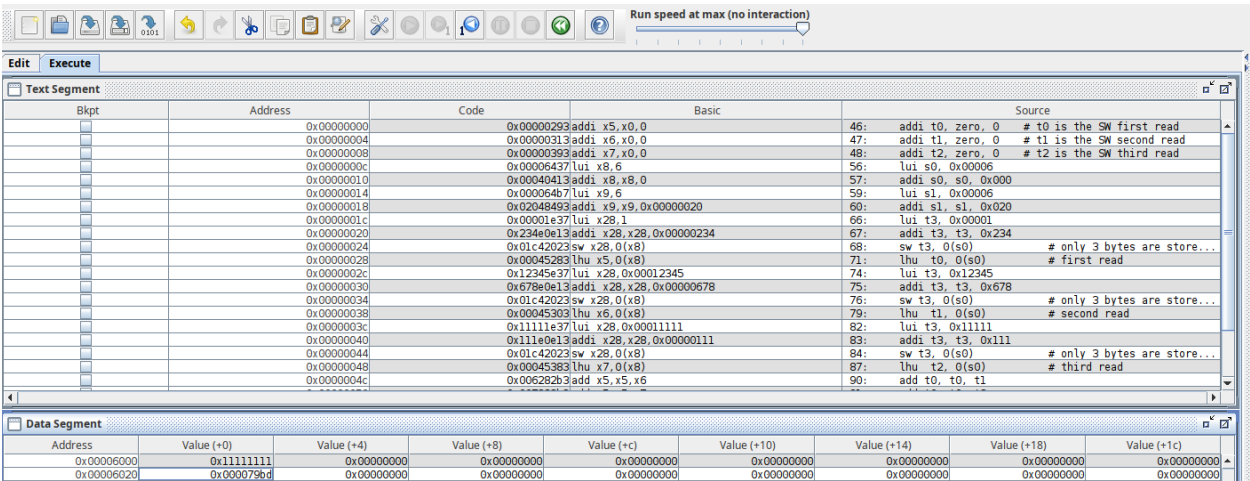


Figure 6: Data segment after executing instructions

Result at LEDS address in 6 matches the answer in 4.

- Test Case 2

```

46 # TC2 : Maxed
47 .equ READ1_H,      0x00005 # switch first read (0x5555)
48 .equ READ1_L,      0x555
49
50 .equ READ2_H,      0x00005 # switch second read (0x5555)
51 .equ READ2_L,      0x555 # sign extended
52
53
54 .equ READ3_H,      0x00005 # switch third read (0x5555)
55 .equ READ3_L,      0x555
56 # answer= 0xFFFF

```

Figure 7: TC2 given input

The screenshot shows a debugger interface with two main panels: 'Text Segment' and 'Data Segment'.

Text Segment: This panel displays a list of instructions with their addresses, codes, basic blocks, and sources. The instructions are as follows:

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	55: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x0000313	addi x6,x0,0	56: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x0000393	addi x7,x0,0	57: addi t2, zero, 0 # t2 is the SW third read
	0x0000000c	0x0000643	lui x8,6	65: lui s0, 0x00006
	0x00000010	0x00049413	addi x8,x8,0	66: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	68: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	69: addi s1, s1, 0x020
	0x0000001c	0x00005e37	lui x28,5	75: lui t3, 0x00005
	0x00000020	0x555e0e13	addi x28,x28,0x00005555	76: addi t3, t3, 0x555
	0x00000024	0x01c42023	sw x28,0(x8)	77: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	80: lhu t0, 0(s0) # first read
	0x0000002c	0x00005e37	lui x28,5	83: lui t3, 0x00005
	0x00000030	0x555e0e13	addi x28,x28,0x00005555	84: addi t3, t3, 0x555
	0x00000034	0x01c42023	sw x28,0(x8)	85: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	88: lhu t1, 0(s0) # second read
	0x0000003c	0x00005e37	lui x28,5	91: lui t3, 0x00005
	0x00000040	0x555e0e13	addi x28,x28,0x00005555	92: addi t3, t3, 0x555
	0x00000044	0x01c42023	sw x28,0(x8)	93: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	96: lhu t2, 0(s0) # third read
	0x0000004c	0x006282b3	add x5,x5,x6	99: add t0, t0, t1

Data Segment: This panel shows the memory values at various addresses:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00006000	0x00005555	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x0000ffff	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 8: Data segment after executing instructions

Result at LEDS address in 8 matches the answer in 7.

• Test Case 3

```

59 # TC3 : Overflow
60 .equ READ1_H,      0x00005 # switch first read (0x5555)
61 .equ READ1_L,      0x555
62
63 .equ READ2_H,      0x00005 # switch second read (0x5555)
64 .equ READ2_L,      0x555 # sign extended
65
66
67 .equ READ3_H,      0x00005 # switch third read (0x5557)
68 .equ READ3_L,      0x557
69 # answer= 0x0001

```

Figure 9: TC3 given input

The screenshot shows a debugger interface with two main panels: 'Text Segment' and 'Data Segment'.

Text Segment: This panel displays a list of instructions with their addresses, codes, basic blocks, and sources. The instructions are as follows:

Bkpt	Address	Code	Basic	Source
	0x00000000	0x0000293	addi x5,x0,0	66: addi t0, zero, 0 # t0 is the SW first read
	0x00000004	0x0000313	addi x6,x0,0	67: addi t1, zero, 0 # t1 is the SW second read
	0x00000008	0x0000393	addi x7,x0,0	68: addi t2, zero, 0 # t2 is the SW third read
	0x0000000c	0x0000643	lui x8,6	76: lui s0, 0x00006
	0x00000010	0x00049413	addi x8,x8,0	77: addi s0, s0, 0x000
	0x00000014	0x000064b7	lui x9,6	79: lui s1, 0x00006
	0x00000018	0x02048493	addi x9,x9,0x00000020	80: addi s1, s1, 0x020
	0x0000001c	0x00005e37	lui x28,5	86: lui t3, 0x00005
	0x00000020	0x555e0e13	addi x28,x28,0x00005555	87: addi t3, t3, 0x555
	0x00000024	0x01c42023	sw x28,0(x8)	88: sw t3, 0(s0) # only 3 bytes are store...
	0x00000028	0x00045283	lhu x5,0(x8)	91: lhu t0, 0(s0) # first read
	0x0000002c	0x00005e37	lui x28,5	94: lui t3, 0x00005
	0x00000030	0x555e0e13	addi x28,x28,0x00005555	95: addi t3, t3, 0x555
	0x00000034	0x01c42023	sw x28,0(x8)	96: sw t3, 0(s0) # only 3 bytes are store...
	0x00000038	0x00045303	lhu x6,0(x8)	99: lhu t1, 0(s0) # second read
	0x0000003c	0x00005e37	lui x28,5	102: lui t3, 0x00005
	0x00000040	0x557e0e13	addi x28,x28,0x0000557	103: addi t3, t3, 0x557
	0x00000044	0x01c42023	sw x28,0(x8)	104: sw t3, 0(s0) # only 3 bytes are store...
	0x00000048	0x00045383	lhu x7,0(x8)	107: lhu t2, 0(s0) # third read
	0x0000004c	0x006282b3	add x5,x5,x6	110: add t0, t0, t1

Data Segment: This panel shows the memory values at various addresses:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00006000	0x00005557	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x00000001	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 10: Data segment after executing instructions

Result at LEDS address in 10 matches the answer in 9.

• Test Case 4

24

Figure 11: TC4 given input

[illegible]

Figure 12: Data segment after executing instruction

Result at LEADS address in 12 matches the answer in 11

• Test Case 5

11

Figure 12: TCE given input

[illegible]

Figure 14: Data segment after executing instruction

Result of LFDG address in 14 matches the answer in 12

• Test Case 6

```

96
97 # TC6 : All highs into full
98 .equ READ1_H,      0x00008 # switch first read
99 .equ READ1_L,      0x000
100
101 .equ READ2_H,      0x00008 # switch second read
102 .equ READ2_L,      0x000
103
104 .equ READ3_H,      0x00008 # switch third read
105 .equ READ3_L,      0x000
106
107 #   answer= 0x8000 (0x18000 as full word though)

```

Figure 15: TC6 given input

Text Segment								
Bkpt	Address	Code	Basic	Source				
	0x00000000	0x00000293	addi x5,x0,0	99: addi t0, zero, 0 # t0 is the SW first read				
	0x00000004	0x00000313	addi x6,x0,0	100: addi t1, zero, 0 # t1 is the SW second read				
	0x00000008	0x00000393	addi x7,x0,0	101: addi t2, zero, 0 # t2 is the SW third read				
	0x0000000c	0x00006437	lui x8,6	109: lui s0, 0x00006				
	0x00000010	0x00040413	addi x8,x8,0	110: addi s0, s0, 0x000				
	0x00000014	0x000064b7	lui x9,6	112: lui s1, 0x00006				
	0x00000018	0x02048493	addi x9,x9,0x00000020	113: addi s1, s1, 0x020				
	0x0000001c	0x00008e37	lui x28,8	119: lui t3, 0x00008				
	0x00000020	0x000e6e13	ori x28,x28,0	120: ori t3, t3, 0x000				
	0x00000024	0x01c42023	sw x28,0(x8)	121: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000028	0x00045283	lhu x5,0(x8)	124: lhu t0, 0(s0) # first read				
	0x0000002c	0x00008e37	lui x28,8	127: lui t3, 0x00008				
	0x00000030	0x000e6e13	ori x28,x28,0	128: ori t3, t3, 0x000				
	0x00000034	0x01c42023	sw x28,0(x8)	129: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000038	0x00045303	lhu x6,0(x8)	132: lhu t1, 0(s0) # second read				
	0x0000003c	0x00008e37	lui x28,8	135: lui t3, 0x00008				
	0x00000040	0x000e6e13	ori x28,x28,0	136: ori t3, t3, 0x000				
	0x00000044	0x01c42023	sw x28,0(x8)	137: sw t3, 0(s0) # only 3 bytes are store...				
	0x00000048	0x00045383	lhu x7,0(x8)	140: lhu t2, 0(s0) # third read				
	0x0000004c	0x006282b3	add x5,x5,x6	143: add t0, t0, t1				
Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x00005000	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x00006020	0x00008000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Figure 16: Data segment after executing instructions

Result at LEDS address in 16 matches the answer in 15.

Assembly Source Code

Listing 1: Question 1 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory three consecutive
4 #           times, summing the unsigned half-word reads and writing
5 #           to the LEDS addr.
6 #
7 # Notes :
8 #   - Assume caller/calle has no need to preserve reg values.
9 #   - KISS method for this assignment
10 #
11 #   - Split 32-bit addresses according to instr imm handling.
12
13
14
15
16 .global main
17
18 .equ SWITCHES_H, 0x11000 # upper 20 bits
19 .equ SWITCHES_L, 0x000 # lower 12 bits
20 .equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
21 .equ SWITCHES_L, 0x000 # lower 12 bits
22
23 .equ LEDS_H, 0x11000 # upper 20 bits
24 .equ LEDS_L, 0x020 # lower 12 bits
25 .equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
26 .equ LEDS_L, 0x020 # lower 12 bits
27
28 # ~~~~~ Test Cases ~~~~~
29 # TC1 : Word Clip
30 .equ READ1_H, 0x00001 # switch first read (0x1234)
31 .equ READ1_L, 0x234
32
33 .equ READ2_H, 0x12345 # switch second read (0x5678)
34 .equ READ2_L, 0x678
35
36 .equ READ3_H, 0x11111 # switch third read (0x1111)
37 .equ READ3_L, 0x111
38 # answer= 0x79BD
39
40 # TC2 : Maxed
41 .equ READ1_H, 0x00005 # switch first read (0x5555)

```



```

42 #.equ READ1_L,      0x555
43
44 #.equ READ2_H,      0x00005 # switch second read (0x5555)
45 #.equ READ2_L,      0x555 # sign extended
46
47 #.equ READ3_H,      0x00005 # switch third read (0x5555)
48 #.equ READ3_L,      0x555
49 # answer= 0xFFFF
50
51 # TC3 : Overflow
52 #.equ READ1_H,      0x00005 # switch first read (0x5555)
53 #.equ READ1_L,      0x555
54
55 #.equ READ2_H,      0x00005 # switch second read (0x5555)
56 #.equ READ2_L,      0x555 # sign extended
57
58 #.equ READ3_H,      0x00005 # switch third read (0x5557)
59 #.equ READ3_L,      0x557
60 # answer= 0x0001
61
62 # TC4 : zero
63 #.equ READ1_H,      0x00000 # switch first read (0)
64 #.equ READ1_L,      0x0
65
66 #.equ READ2_H,      0x00000 # switch second read (0)
67 #.equ READ2_L,      0x0 # sign extended
68
69 #.equ READ3_H,      0x00000 # switch third read (0)
70 #.equ READ3_L,      0x0
71 # answer= 0x0000
72
73 # TC5 : MAXXXED
74 #.equ READ1_H,      0x0000F # switch first read (0xFFFF)
75 #.equ READ1_L,      -1
76
77 #.equ READ2_H,      0xFFFFF # switch second read (0xFFFF)
78 #.equ READ2_L,      -1
79
80 #.equ READ3_H,      0x00000 # switch third read (0)
81 #.equ READ3_L,      0
82 # answer= 0xFFFE
83
84 # TC6 : All highs into full
85 .equ READ1_H,      0x00008 # switch first read
86 .equ READ1_L,      0x000
87
88 .equ READ2_H,      0x00008 # switch second read
89 .equ READ2_L,      0x000
90
91 .equ READ3_H,      0x00008 # switch third read
92 .equ READ3_L,      0x000
93 # answer= 0x8000 (0x18000 as full word though)
94 # ~~~~~
95
96 main:
97
98 # ~~~~ register init ~~~~
99 addi t0, zero, 0 # t0 is the SW first read
100 addi t1, zero, 0 # t1 is the SW second read
101 addi t2, zero, 0 # t2 is the SW third read
102 # t3 is a dummy variable for testing
103
104 # s0 contains the SWITCH addr
105 # s1 contains the LEDS addr
106
107
108 # ~~~~ load addr into registers ~~~~
109 lui s0, SWITCHES_H
110 addi s0, s0, SWITCHES_L
111
112 lui s1, LEDS_H
113 addi s1, s1, LEDS_L
114
115
116 # ~~~~ read SWITCHES and sum ~~~~
117
118 # ~~ behind the scenes : switch mem changes value ~~
119 lui t3, READ1_H
120 ori t3, t3, READ1_L
121 sw t3, 0(s0) # only 3 bytes are stored (0x234)
122 # ~~~~~
123
124 lhu t0, 0(s0) # first read
125
126 # ~~ behind the scenes : switch mem changes value ~~
127 lui t3, READ2_H
128 ori t3, t3, READ2_L
129 sw t3, 0(s0) # only 3 bytes are stored (0x678)
130 # ~~~~~
131
132 lhu t1, 0(s0) # second read

```

```
133
134 # ~~~~ behind the scenes : switch mem changes value ~~~~
135 lui t3, READ3_H
136 ori t3, t3, READ3_L
137 sw t3, 0(s0)      # only 3 bytes are stored (0x111)
138 # ~~~~~
139
140 lhu t2, 0(s0)      # third read
141
142 # ~~~~ sum values ~~~~
143 add t0, t0, t1
144 add t0, t0, t2
145
146
147 # ~~~~ Write the unsigned half word to LEDS mem addr
148 sh t0, 0(s1)
149
150
151 # ~~~~ return with exit status ~~~~
152 li a7, 10  # load system call number for exit()
153 li a0, 0    # Load the exit status code ( EXIT_SUCCESS )
154 ecall      # Make the system call
```

Part 2

"Read an input from SWITCHES (address 0x11000000). Assume the input is a 16-bit signed value in 2's complement (RC). Change the sign of the input and output the result to LEDS (address 0x11000020). The result should also be a 16-bit signed value in 2's complement (RC)."

Flowchart

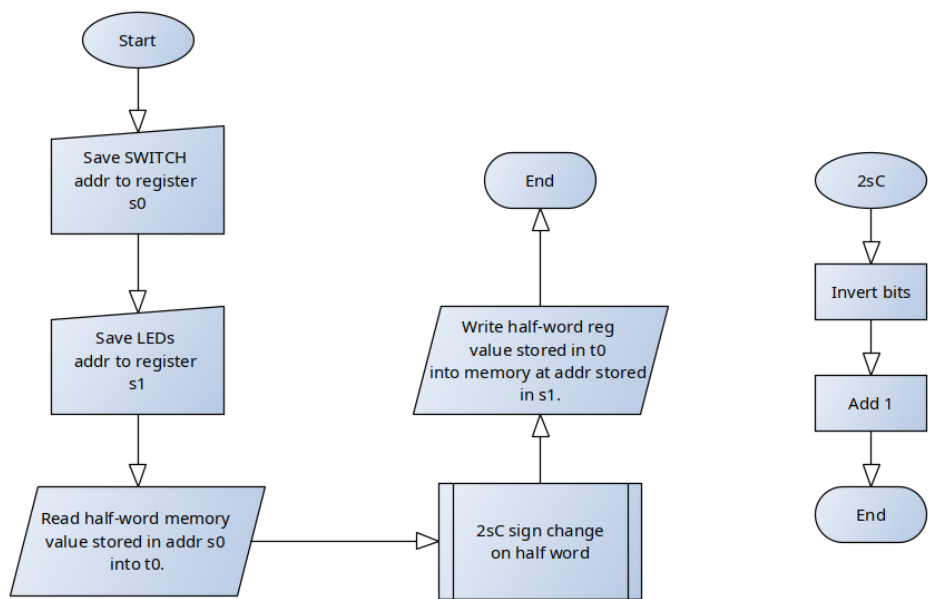


Figure 17: Flow Chart for Part 2

Verification Simulations

Test cases passed with definitions defined in code below.

```
16 # TC1 : zero
17 #A: .half 0
18 # answer= 0x0000
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000000
0x00006020	0x00000000

Figure 18: IO and Data Segment of TC1

```
21 # TC2 : positive
22 #A: .half 0x5
23 # answer= 0xFFFFB (-5)
```

Data Segment	
Address	Value (+0)
0x00006000	0x00000005
0x00006020	0x0000ffffb

Figure 19: IO and Data Segment of TC2

25	# TC3 : negative		
26	#A: .half 0xFFFB	# -5	
27	# answer= 0x0005	# 5	
28			

Data Segment	
Address	Value (+0)
0x00006000	0x0000fffb
0x00006020	0x00000005

Figure 20: IO and Data Segment of TC3

30	# TC4 : positive		
31	#A: .half 0x5678	# 22136	
32	# answer= 0xA988	# -22136	

Data Segment	
Address	Value (+0)
0x00006000	0x00005678
0x00006020	0x0000a988

Figure 21: IO and Data Segment of TC4

34	# TC5 : reverse		
35	#A: .half 0xA988		
36	# answer= 0x5678		
37			

Data Segment	
Address	Value (+0)
0x00006000	0x0000a988
0x00006020	0x00005678

Figure 22: IO and Data Segment of TC5

39	# TC6 : negative		
40	#A : .half 0xFFFF #(-1)		
41	# answer= 1		
42			

Data Segment	
Address	Value (+0)
0x00006000	0x0000ffff
0x00006020	0x00000001

Figure 23: IO and Data Segment of TC6

43	# TC7 : negative 2		
44	#A : .half 0xFFFE #(-2)		
45	# answer= 2		
46			

Data Segment	
Address	Value (+0)
0x00006000	0x0000fffe
0x00006020	0x00000002

Figure 24: IO and Data Segment of TC7

48	# TC7 : negative 32 bit		
49	#A : .word 0x89ABCDEF	#(0xCDEF = -12817)	
50	# answer= 0x3211	# 12817	
51			

Data Segment	
Address	Value (+0)
0x00006000	0x89abcdef
0x00006020	0x00003211

Figure 25: IO and Data Segment of TC8

Assembly Code

Listing 2: Question 2 of SW1

```

1 # Author : WhisperingRock
2 #
3 # Purpose : Read from the SWITCHES addr in memory a unsigned half-word
4 #           changing the sign, then writing to the LEDS addr.
5 #
6 # Notes :
7 #     - Assume caller/callee has no need to preserve reg values.
8 #       KISS method for this assignment
9 #
10 #     - Split 32-bit addresses according to instr imm handling.
11
12 .data
13 # ~~~~~ Test Cases ~~~~~
14 # TC1 : zero
15 #A: .half 0
16 # answer= 0x0000
17
18 # TC2 : positive
19 #A: .half 0x5
20 # answer= 0xFFFB (-5)
21
22 # TC3 : negative
23 #A: .half 0xFFFB # -5
24 # answer= 0x0005 # 5
25
26 # TC4 : positive
27 #A: .half 0x5678 # 22136

```

```

28 # answer= 0xA988      # -22136
29
30 # TC5 : reverse
31 #A: .half 0xA988
32 # answer= 0x5678
33
34 # TC6 : negative
35 #A : .half 0xFFFF #(-1)
36 # answer= 1
37
38 # TC7 : negative 2
39 #A : .half 0xFFFE #(-2)
40 # answer= 2
41
42 # TC7 : negative 32 bit
43 #A : .word 0x89ABCDEF    #(0xCDEF = -12817)
44 # answer= 0x3211      # 12817
45
46 # ~~~~~
47
48
49 .global _start
50 .text
51 #.equ SWITCHES_H, 0x11000 # upper 20 bits
52 #.equ SWITCHES_L, 0x000 # lower 12 bits
53 #.equ SWITCHES_H, 0x00006 # upper 20 bits (data seg at 0x00006000)
54 #.equ SWITCHES_L, 0x000 # lower 12 bits
55
56 #.equ LEDS_H, 0x11000 # upper 20 bits
57 #.equ LEDS_L, 0x020 # lower 12 bits
58 #.equ LEDS_H, 0x00006 # upper 20 bits (data seg at 0x00006020)
59 #.equ LEDS_L, 0x020 # lower 12 bits
60
61
62
63 _start:
64
65 # ~~~~ register init ~~~~
66 addi t0, zero, 0 # t0 is the SW first read
67 # t3 is a dummy variable for testing
68
69 # s0 contains the SWITCH addr
70 # s1 contains the LEDS addr
71
72 # ~~~~ load addr into registers ~~~~
73 lui s0, SWITCHES_H
74 addi s0, s0, SWITCHES_L
75
76 lui s1, LEDS_H
77 addi s1, s1, LEDS_L
78
79
80 # ~~~~ read SWITCHES and sum ~~~~
81
82 # ~~ behind the scenes : switch value ~~
83 lw t3, A
84 sw t3, 0(s0)
85 # ~~~~~
86
87 lhu t0, 0(s0) # first read
88
89
90 # ~~~~ invert the sign (2sC) ~~~~
91 not t0, t0
92 addi t0, t0, 1
93
94 # ~~~~ Write the unsigned half word to LEDS mem addr
95 sh t0, 0(s1)
96
97
98 # ~~~~ return with exit status ~~~~
99 li a7, 10 # load system call number for exit()
100 li a0, 0 # Load the exit status code ( EXIT_SUCCESS )
101 ecall # Make the system call

```

0.3 Appendix